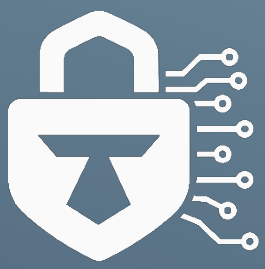


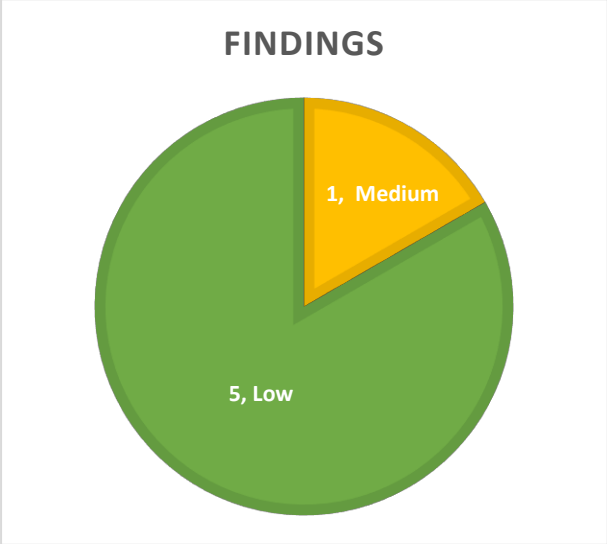
TrustSec

Smart Contract Audit

Morpho – Midnight

10/07/2026

Executive summary

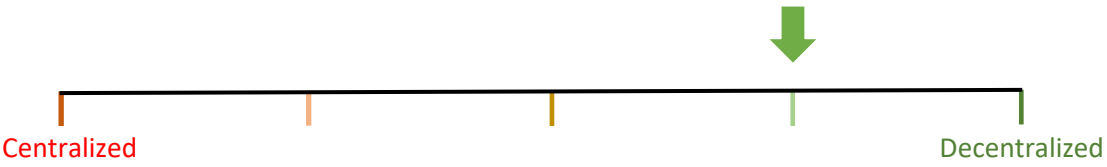


Category	Lending
Audited file count	13
Auditor	Trust HollaDieWaldfee
Time period	20/05/2026 - 05/06/2026

Findings

Severity	Total	Fixed	Acknowledged
High	0	0	0
Medium	1	1	0
Low	5	4	1

Centralization score



Signature

Digitally signed by Trust
Date: 2026.07.15 20:36:42+03'00'

Document properties

Versioning

Version	Date	Description
0.1	05/06/2026	Client report
0.2	10/07/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- src/libraries/ConstantsLib.sol
- src/libraries/EventsLib.sol
- src/libraries/IdLib.sol
- src/libraries/SafeTransferLib.sol
- src/libraries/TickLib.sol
- src/libraries/UtilsLib.sol
- src/periphery/ConsumableUnitsLib.sol
- src/periphery/EcrecoverAuthorizer.sol
- src/periphery/TakeAmountsLib.sol
- src/ratifiers/libraries/HashLib.sol
- src/ratifiers/EcrecoverRatifier.sol
- src/ratifiers/SetterRatifier.sol
- src/Midnight.sol

Repository details

- **Repository URL:** https://github.com/morpho-org/midnight
- **Commit hash:** 6783b978361f330de292715abf72f5f38bcb8523
- **Mitigation hash:** e6f2bf28400d8215a533f713081a6f68ca121941

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under TrustSec. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	**Good**	Midnight is a minimal and simple lending primitive.
Documentation	**Excellent**	Project is well documented with a whitepaper and code comments.
Best practices	**Excellent**	Project adheres to best practices.
Centralization risks	**Excellent**	Protocol is permissionless, only using privileged roles for fee settings and tick spacings. Fees have tight upper bounds.

Findings

Medium severity findings

TRST-M-1 Borrowers can create dust debt and collateral that is unprofitable to liquidate

- **Category:** Logical issues
- **Source:** Midnight.sol
- **Status:** Fixed

Description

The liquidation documentation describes **rcfThreshold** as a mitigation for liquidations that are too small compared to gas cost:

```
/// @dev The RCF is deactivated for small collateral amount, essentially to mitigate
issues with liquidations that are
/// too small compared to the gas cost. More precisely, it is deactivated if the
liquidation could leave a collateral
/// with a value that would not be enough to repay rcfThreshold units.
```

With regards to **rcfThreshold,** it has been acknowledged by the protocol that liquidators may leave less than **rcfThreshold** collateral in the liquidated account. This is done for practical reasons such that liquidators don't have to repay exact amounts which could delay liquidations. However, liquidations are not the only source of dust debts and dust collateral. The protocol allows economically small debt and collateral positions to be created by borrowers.

The *supplyCollateral()* function accepts any **assets** amount and does not enforce a minimum collateral amount or minimum collateral value:

```
function supplyCollateral(Market memory market, uint256 collateralIndex, uint256
assets, address onBehalf)
    external
{
    ...
    _position.collateral[collateralIndex] = UtilsLib.toUint128(oldCollateral +
assets);
    ...
}
```

Similarly, *take()* can create borrower debt for any positive **units** amount that passes the offer and market checks:

```
uint256 sellerDebtIncrease = units - sellerCreditDecrease;
...
sellerPos.debt += UtilsLib.toUint128(sellerDebtIncrease);
```

Users can also reduce larger positions down to dust through partial repayments or collateral withdrawals. *repay()* only subtracts the specified **units** from the borrower's debt, and *withdrawCollateral()* only checks that the resulting position is healthy:

This means that positions with very small debt, very small collateral, or both can exist before they ever enter the liquidation path. This introduces incoherence with regards to achieving the stated goals of the **rcfThreshold**.

Additionally, to maximize the impact of dust positions, an attacker can intentionally split exposure into many small positions while transaction costs are low. If those positions later become liquidatable when gas costs are higher, rational liquidators ignore them because the available debt repayment and collateral seizure are too small to cover costs. Overall, this significantly increases the risk of bad debt.

Recommended mitigation

It is recommended to define explicit minimum economic sizes for debt and collateral positions and enforce them consistently when collateral is supplied / withdrawn and debt created / repaid. For debt, this can be defined on the market-level and for collateral it can be defined per market and collateral. This also allows individual markets to disable the setting if desired.

For liquidations, it should be documented that there is a risk of liquidators leaving less than **rcfThreshold** worth of collateral.

Team response

Fixed by documenting the behavior in commit 40e94ff.

Low severity findings

TRST-L-1 Bilaterally shared offers can be taken by any user

- **Category:** Logical issues
- **Source:** Midnight.sol
- **Status:** Fixed

Description

The whitepaper that has been provided as documentation for the audit describes offer distribution as follows:

"The protocol is intentionally agnostic to how offers are discovered. Offers can exist as offchain signatures whose distribution may be public, bilateral, or anything in between."

However, the protocol does not provide a way for the maker to restrict an offer to a specific taker. In *take()*, the only taker authorization check ensures that the caller is allowed to act for the chosen **taker**:

```
require(taker == msg.sender || isAuthorized[taker][msg.sender], TakerUnauthorized());
```

This protects the taker from unauthorized delegation, but it does not let the maker restrict who can take the offer. The ratifier does also not accept the taker as a parameter:

```
function isRatified(Offer memory offer, bytes memory ratifierData) external view
returns (bytes32);
```

As a result, if a bilaterally shared offer and its ratifier data are observed in the public mempool, any third party can copy the data and take the offer first by setting themselves as **taker**. This violates the intended bilateral use case.

While a market-level **enterGate** can restrict which accounts may increase credit or debt, and private transaction submission can reduce the likelihood of copying, these are not equivalent to an offer-level taker restriction on the protocol level. A market gate is broader than a bilateral route, and private mempools introduce their own availability and centralization assumptions, while the whitepaper suggests this functionality should be part of the protocol's native functionality.

Recommended mitigation

It is recommended to extend the **Offer** struct or *IRatifier* interface such that offers can commit to a specific taker. Alternatively, the whitepaper and related documentation may be adjusted to remove this use case if it is not considered necessary.

Team response

Fixed in commit e6f2bf2.

Mitigation review

Verified, the *IRatifier* interface has been expanded to include the **taker** address.

TRST-L-2 Takers can leave dust offer capacity

- **Category:** Griefing issues
- **Source:** Midnight.sol
- **Status:** Fixed

Description

Offers are protected by consumption caps, but there is no requirement that a partial fill either consumes the offer completely or leaves a meaningful residual amount for future takers.

In *take()*, **consumed** is increased by either the filled asset amount or the filled unit amount, and the only cap check is that the new value does not exceed **maxAssets** or **maxUnits**:

```
if (offer.maxAssets > 0) {
    newConsumed = consumed[offer.maker][offer.group] += offer.buy ? buyerAssets :
sellerAssets;
    require(newConsumed <= offer.maxAssets, ConsumedAssets());
} else {
    newConsumed = consumed[offer.maker][offer.group] += units;
    require(newConsumed <= offer.maxUnits, ConsumedUnits());
}
```

A taker can therefore intentionally fill an amount that leaves a very small residual capacity in the offer's **group**. This residual may be too small to be worth routing, gas, or operational overhead, but it still prevents the maker's quoted liquidity from being fully consumed at the intended rate.

The impact is limited because the maker's cap is not exceeded and the remaining amount is still technically available. However, it can create a persistent gap between the amount of liquidity the maker intended to expose and the amount that is practically fillable by future takers.

Recommended mitigation

Consider adding a minimum residual fill rule. For example, a fill could be required to satisfy one of the following:

```
newConsumed == max || max - newConsumed >= minResidual
```

The **minResidual** value could be a market-level configuration or a global default expressed as a percentage of the relevant cap, with a privileged role able to adjust it for markets where token value or gas assumptions make the default inappropriate. If no such rule is added, it should be documented that takers can leave economically unfillable dust capacity in offer groups.

Team response

Fixed by documenting the risk in PR1043.

TRST-L-3 Deauthorization does not revoke recursively set authorities or ratified roots

- **Category:** Access control issues
- **Source:** Midnight.sol, SetterRatifier.sol
- **Status:** Acknowledged

Description

Midnight authorizations are absolute permissions. Any authorized address can call functions on behalf of the authorizer, including *setIsAuthorized()* itself:

```
function setIsAuthorized(address authorized, bool newIsAuthorized, address onBehalf)
external {
    require(onBehalf == msg.sender || isAuthorized[onBehalf][msg.sender],
Unauthorized());
    isAuthorized[onBehalf][authorized] = newIsAuthorized;
    emit EventsLib.SetIsAuthorized(msg.sender, onBehalf, authorized,
newIsAuthorized);
}
```

As a result, if user **A** authorizes address **B**, **B** can authorize address **C** on behalf of **A**. If **A** later removes **B** from the authorization mapping, the authorization granted to **C** remains active. There is no parent-child relationship between authorizations and no automatic revocation of permissions that were installed through a deauthorized address.

In addition to breaking a user's mental model for revocation, this authorization model is contradictory. It is intended that authorization is absolute. But if it is intended to be absolute, then deauthorizing is unnecessary (and potentially misleading) functionality.

As a practical example, a user may authorize a third-party integration while trading with a small account balance, then later remove that integration before increasing their exposure. If the integration previously installed another authorized address, that address can continue acting on behalf of the user after the visible integration has been removed. The scenario demonstrates that on-chain trust on a 3rd-party can be limited in exposure through time segmentation, and removal of that trust should be conservative and err on the side of caution.

A related persistence issue exists in *SetterRatifier*. An authorized address can ratify roots for a maker:

```
function setIsRootRatified(address maker, bytes32 root, bool newIsRootRatified)
public {
    require(maker == msg.sender || IMidnight(MIDNIGHT).isAuthorized(maker,
msg.sender), Unauthorized());
    isRootRatified[maker][root] = newIsRootRatified;
    emit SetIsRootRatified(msg.sender, maker, root, newIsRootRatified);
}
```

Once the root is stored, *isRatified()* only checks the stored root approval:

```
require(isRootRatified[offer.maker][root], NotRatified());
```

Therefore, roots ratified by **B** for **A** remain usable after **A** deauthorizes **B**, unless **A** also identifies and explicitly un-ratifies those roots. This differs from *EcrecoverRatifier*, where an offer signed by **B** stops being valid after **B** is no longer authorized because the signer authorization is checked at fill time:

```
require(_signer == offer.maker || IMidnight(MIDNIGHT).isAuthorized(offer.maker,
_signer), Unauthorized());
```

The impact is limited by the fact that an authorized address is already highly trusted and can directly act maliciously while authorized. However, the persistence of secondary authorizations and ratified roots can extend that authority beyond the moment where the user believes it has been revoked.

Recommended mitigation

It is recommended to implement stronger revocation semantics by changing the authorization model so that delegated permissions can be revoked recursively. For *SetterRatifier*, consider storing the address that ratified each root and requiring that address to still be authorized at fill time, or binding root approvals to a maker authorization epoch that can be incremented to invalidate previously ratified roots.

Team response

Acknowledged.

TRST-L-4 Continuous fee changes cannot be protected against in *Midnight* callbacks

- **Category:** Race conditions
- **Source:** Midnight.sol
- **Status:** Fixed

Description

The **feeSetter** can update a market's **continuousFee**. Existing lenders are protected because their already created **pendingFee** is fixed, but new credit created by *take()* uses the live **continuousFee** at execution time.

In *take()*, the continuous fee prepaid by newly created buyer credit is computed from the current market state:

```
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivDown(_marketState.continuousFee *
timeToMaturity, WAD));
```

If the **feeSetter** raises **continuousFee** before a user's transaction is included, the transaction can still execute but create a larger **pendingFee** than the user expected. The user's face value is **credit - pendingFee**, so the user effectively receives less value for the same quoted trade.

The impact is bounded because **continuousFee** is capped by **MAX_CONTINUOUS_FEE**, which is 1% per year. For a one-year position, the unexpected fee increase is therefore capped at about 1% of the newly created credit. However, this can still be a material on-chain result for large trades or integrations that rely on exact face-value accounting.

The protocol provides callbacks that can be used to validate execution conditions atomically, but the current callback data is insufficient to fully validate the continuous-fee effect. *IBuyCallback.onBuy()* receives **units** and **pendingFeeIncrease**:

```
function onBuy(
    bytes32 id,
    Market memory market,
    uint256 buyerAssets,
    uint256 units,
    uint256 pendingFeeIncrease,
    address buyer,
    bytes memory data
) external returns (bytes32);
```

However, **pendingFeeIncrease** is computed from **buyerCreditIncrease**, not from all **units**. Some of the filled **units** may repay existing buyer debt instead of creating new credit:

```
uint256 buyerCreditIncrease = UtilsLib.zeroFloorSub(units, buyerPos.debt);
```

Therefore, the callback cannot determine from its parameters alone whether the observed **pendingFeeIncrease** was caused by the expected credit increase at the expected continuous-fee rate, or by a different credit/debt split and fee rate.

The **Take** event includes **buyerCreditIncrease** and **sellerCreditDecrease**, but events are not available to callback code during the transaction. A callback could try to reconstruct the missing values by reading additional on-chain state, but that adds complexity and is not part of the explicit callback interface.

By contrast, the **tradingFee** is always paid by the taker to protect the maker, and the taker can use the callback to check the effective trade price as documented in *Midnight*.

Recommended mitigation

Include the credit/debt split in the callback parameters. For example, pass **buyerCreditIncrease** to *onBuy()* and **sellerCreditDecrease** to *onSell()*:

```
IBuyCallback(buyerCallback).onBuy(
    id,
    offer.market,
    buyerAssets,
    units,
    buyerPendingFeeIncrease,
    buyerCreditIncrease,
    buyer,
    buyerCallbackData
);

ISellCallback(sellerCallback).onSell(
    id,
    offer.market,
    sellerAssets,
    units,
    sellerPendingFeeDecrease,
    sellerCreditDecrease,
    seller,
    receiver,
    sellerCallbackData
);
```

An alternative mitigation is to add the **pendingFee** limits directly to the *take()* interface and the **Offer** struct. But that may be undesirable and against the intentional simplicity of *Midnight*.

Team response

Fixed in PR994 by adding **continuousFeeCap** to the **Offer** struct. Protecting the taker against fee changes is left to integrators.

Mitigation review

Verified, the **continuousFeeCap** for makers has been correctly implemented.

TRST-L-5 Enter gate relies on fragile view declaration for reentrancy protection, risking permanent loss of funds

- **Category:** Reentrancy issues
- **Source:** Midnight.sol
- **Status:** Fixed

Description

Midnight.take() calls the market **enterGate** after it has read position storage, updated some positions, and computed intermediate accounting values such as **buyerCreditIncrease**, **sellerCreditDecrease**, **buyerPendingFeeIncrease**, and **sellerPendingFeeDecrease**:

```
if (hasCredit(id, buyer) || units > buyerPos.debt) _updatePosition(offer.market, id,
buyer);
if (hasCredit(id, seller)) _updatePosition(offer.market, id, seller);

uint256 buyerCreditIncrease = UtilsLib.zeroFloorSub(units, buyerPos.debt);
uint256 sellerCreditDecrease = UtilsLib.min(units, sellerPos.credit);
uint256 sellerDebtIncrease = units - sellerCreditDecrease;
```

The gate checks are then performed before the position storage writes:

```
require(
    offer.market.enterGate == address(0) || buyerCreditIncrease == 0
        || IEnterGate(offer.market.enterGate).canIncreaseCredit(buyer),
    BuyerGatedFromIncreasingCredit()
);
require(
    offer.market.enterGate == address(0) || sellerDebtIncrease == 0
        || IEnterGate(offer.market.enterGate).canIncreaseDebt(seller),
    SellerGatedFromIncreasingDebt()
);

buyerPos.debt -= UtilsLib.toUint128(units - buyerCreditIncrease);
buyerPos.pendingFee += buyerPendingFeeIncrease;
buyerPos.credit += UtilsLib.toUint128(buyerCreditIncrease);
```

This is safe in the current implementation because *IEnterGate.canIncreaseCredit()* and *IEnterGate.canIncreaseDebt()* are declared **view**:

```
interface IEnterGate {
    function canIncreaseCredit(address account) external view returns (bool);
    function canIncreaseDebt(address account) external view returns (bool);
}
```

It has been determined that this is unintended and the code should be safe without the **view** declaration present since the declaration may theoretically be changed from **view** to non-**view** as future use cases arise. Moreover, the code is also vulnerable for all Solidity versions that support the **view** keyword before v0.5.0, when the compiler started to emit **STATICCALL** for this use case**.**

A POC demonstrates that changing the gate interface or replacing the **STATICCALL** behavior with a normal call creates a reentrancy attack vector that causes permanent loss of funds. It performs the following steps:

1. **lender** starts with 100 credit and **lastLossFactor = 0**.
2. The oracle is crashed, making **borrower** liquidatable with bad debt.
3. **lender** buys 50 additional units.
4. During take, the buyer is updated before the gate call, pinning **lastLossFactor** to the old value.
5. A malicious non-view gate reenters **liquidate(..., 0, 0, ...)**, realizing bad debt and advancing the market **lossFactor**.
6. Control returns to take, which adds the new 50 credit while the buyer's **lastLossFactor** remains stale.
7. As a result, the buyer's new credit is later slashed as if it existed before the bad debt, creating an excess of slashed funds that are permanently inaccessible and stuck in

Midnight. The buyer should end up with 51 credit, but due to the reentrancy ends up with 2 credit. This is a permanent loss of funds for the buyer.

Recommended mitigation

It is recommended to call the *IEnterGate* hooks at the end of *take()*, before the buyer callback. This ensures protection against reentrancies even if the **view** declaration is removed or if the calls are compiled with older Solidity versions.

Team response

Fixed in PR977. The **view** modifier is not going to change. However, a small refactor in *take()* still changes the order to avoid the **view** call reading a slightly inconsistent state w.r.t. **consumed**.

Mitigation review

To avoid the gate reading an already updated **consumed** value, **consumed** is now set after the gate is called. The **view** modifier is understood as critical and as the only defense to protect against the reentrancy attack in the POC above.

Additional recommendations

TRST-R-1 Document that post-maturity bad debt accounting uses maxLif

The *liquidate()* function computes bad debt by discounting every collateral at the collateral's terminal **maxLif**, including when the caller selects the post-maturity **healthyPath**. In that path, the active liquidation incentive factor starts at **WAD** at maturity and ramps toward **maxLif** over **TIME_TO_MAX_LIF**.

This creates an asymmetry between bad debt accounting and collateral seizure. The protocol socializes bad debt using the stricter terminal **maxLif**, while the selected post-maturity liquidation path may still seize collateral using a lower current liquidation incentive. This can make a liquidation realize more bad debt than would be implied by the current post-maturity liquidation terms alone.

This behavior is a design choice to prioritize solvency. It preserves the property that any liquidation realizes all bad debt measured under the terminal liquidation incentive, rather than allowing softer post-maturity liquidations to leave additional bad debt to be discovered by a later **!healthyPath** liquidation or a **healthyPath** liquidation with increased **lif**.

However, the tradeoff is subtle and may be surprising to lenders, liquidators, and integrators. It is recommended to document this explicitly in the liquidation comments section of *Midnight* and in any external integration documentation. The documentation should explain that bad debt realization is intentionally based on terminal **maxLif** for solvency accounting.

TRST-R-2 Document that positive rebasing token surplus is lost

Midnight documents that a supported token's balance in the protocol should only decrease on *transfer()* and *transferFrom()*. This allows positive rebasing tokens at the token-safety level, since an upward rebase increases *Midnight*'s raw token balance without being caused by a token transfer.

However, this additional balance is not reflected in protocol accounting and can't be withdrawn. Users can only withdraw or claim amounts that are represented by protocol accounting, and there is no general sweep function for surplus token balances. Therefore, upward rebases can leave extra token balances permanently stuck in *Midnight*.

It is recommended to explicitly document that positive rebasing tokens may create inaccessible surplus balances, even if they otherwise satisfy the current token-safety requirements.

TRST-R-3 Document *ecrecover()* signature malleability

EcrecoverRatifier and *EcrecoverAuthorizer* verify signatures with the raw *ecrecover()* precompile. Raw *ecrecover()* accepts malleable signatures which common libraries such as OZ's *ECDSA* reject.

The same signer and digest can have more than one valid signature representation. This does not create a security issue in the current implementation because neither contract relies on signature bytes being unique. *EcrecoverRatifier* authorizes a **root**, while *EcrecoverAuthorizer* uses an explicit **nonce** for replay protection.

It is recommended to document that raw *ecrecover()* is used intentionally and that signature uniqueness is not a security assumption.

TRST-R-4 Document the theoretical max collaterals restriction due to EIP-170 limit

IdLib stores a market by deploying **abi.encode(market)** as runtime bytecode with **CREATE2**. This runtime bytecode is subject to the EIP-170 maximum contract size of 24,576 bytes.

The encoded size of a market with **N** collateral entries is **256 + 128 * N** bytes. The first 32 bytes are the ABI offset to the dynamic struct body, the market body then contains six 32-byte head words, the dynamic **collateralParams** array length, and four 32-byte fields per collateral entry.

With the current **MAX_COLLATERALS** value of 128, the encoded market size is 16,640 bytes, leaving significant headroom below the EIP-170 limit. However, the largest value that fits is 190 collaterals:

```
N = 128 -> 16,640 bytes
N = 190 -> 24,576 bytes
N = 191 -> 24,704 bytes
```

Therefore, if **MAX_COLLATERALS** were ever increased above 190, markets at the configured maximum collateral count would become uncreatable. The *CREATE2* operation in *IdLib* would fail because the runtime bytecode exceeds the EIP-170 limit, causing *touchMarket()* to revert.

It is recommended to document the coupling between **MAX_COLLATERALS** and the contract size limit.

TRST-R-5 Document EIP-7702 authorization risk

Midnight uses the **isAuthorized** mapping to grant broad permissions. An authorized address can act on behalf of the authorizer across all protocol functions, and the same authorization mechanism is also used for ratifiers and other scoped helper contracts.

This creates an important integration assumption. When a user authorizes a helper contract, they are trusting the behavior of the address they authorized. With EIP-7702, an EOA can temporarily or repeatedly install delegated code while still being able to originate transactions. Therefore, an address may appear contract-like and provide scoped behavior but still be controlled by an EOA.

It is recommended to document that scoped ratifier and helper contracts should not be deployed or represented through EIP-7702 delegated EOAs unless users explicitly understand that the authorized address remains under EOA control and its delegated behavior can change.

TRST-R-6 *SetterRatifier* does not commit ratified roots to a tree height

EcrecoverRatifier signs a typed EIP-712 tree commitment that binds the Merkle root to the tree height by including *HashLib.offerTreeTypeHash()* with **proof.length** in the signed struct hash. This also enforces the practical height cap implemented by *offerTreeTypeHash()*.

SetterRatifier instead stores ratification directly on the raw **root** through **isRootRatified**. During *isRatified()*, it verifies the Merkle proof against the raw **root** and does not bind the ratification to the proof length or typed tree commitment.

There is no security impact since reusing a ratified raw **root** with a different proof length would require finding a different-height Merkle tree with the same root, or a related collision between offer leaf hashes and internal node hashes. However, this creates a semantic inconsistency between *SetterRatifier* and *EcrecoverRatifier*, and it means *SetterRatifier* does not preserve the same typed tree commitment convention.

It is recommended to either document this difference explicitly or align *SetterRatifier* with *EcrecoverRatifier* by ratifying the typed tree commitment rather than the raw **root**. For example, *SetterRatifier* could store and check:

```
bytes32 typedRoot = keccak256(abi.encode(HashLib.offerTreeTypeHash(proof.length),
root));
require(isRootRatified[offer.maker][typedRoot], NotRatified());
```

This would also make the maximum supported tree height explicit through *HashLib.offerTreeTypeHash()*.

TRST-R-7 Return *multicall()* results

Midnight.multicall() executes each **calls** item through *delegatecall()* and correctly bubbles up reverts, but it does not return the successful return data to the caller.

This limits composability for integrations that batch calls and want to consume return values from functions such as *take()* or *liquidate()*. Therefore, it is recommended to return an array of return data from *multicall()*, matching the common multicall interface pattern:

```
function multicall(bytes[] calldata calls) external returns (bytes[] memory results)
```

TRST-R-8 Skip *withdrawCollateral()* health check while liquidation is locked

The *withdrawCollateral()* function always calls *isHealthy()* after decreasing an account's collateral. This is stricter than the final seller check in *take()*, which explicitly treats **liquidationLocked(id, seller)** as an allowed transient state during callback execution.

This difference matters when *withdrawCollateral()* is called during a *take()* callback. In that situation, the health check is not necessary because the seller's final health is checked after the callback once the lock is cleared. Keeping the health check inside *withdrawCollateral()* therefore creates unnecessary dependencies during a transient state that *Midnight* already models separately.

Consider making *withdrawCollateral()* consistent with *take()* by skipping the immediate health check when **liquidationLocked(id, onBehalf)** is **true**:

```
require(liquidationLocked(id, onBehalf) || isHealthy(market, id, onBehalf),
UnhealthyBorrower());
```

TRST-R-9 Document that outer context can revert inner context execution

By allowing callbacks, an outer execution context can change *Midnight*'s state such that an inner execution context reverts. For example, an attacker-controlled outer call can use *flashLoan()* to temporarily transfer out all of *Midnight*'s balance of a loan token or collateral token. While those tokens are out of the contract, an inner victim call to *withdraw()*, *withdrawCollateral()* or *liquidate()* can revert when *Midnight* attempts to transfer funds to the victim. The flash loan can later be repaid before the outer call finishes, so the failure is caused by temporary context rather than by a persistent protocol state change.

This matters for account abstraction, routers, solvers, keepers, or other execution systems where a user's intended *Midnight* interaction may be triggered inside a context controlled by another party.

It is recommended to document that *Midnight* calls should be treated as context-sensitive. Integrators should avoid executing *Midnight* user interactions inside untrusted outer calls.

TRST-R-10 Add a loss-factor safety margin to *take()*

Midnight documents that if a market loses almost all of its value to bad debt over its lifetime, the accounting of the **lossFactor** can become extremely imprecise and the market may stop functioning properly. The documentation also notes that *take()* reverts once the loss factor is maxed out.

In code, *take()* only rejects the exact terminal value:

```
require(_marketState.lossFactor < type(uint128).max, MarketLossFactorMaxedOut());
```

It is recommended to add a more conservative safety margin directly in *take()*, so new fills are rejected before **lossFactor** enters the range where accounting is known to become extremely imprecise. For example, the protocol could introduce a constant threshold and use it instead of the exact terminal value.

TRST-R-11 Document frontrunning of signed authorizations

EcrecoverAuthorizer.setIsAuthorized() allows anyone to submit a valid signed **Authorization** on behalf of the **authorizer**. This is useful for gasless authorization flows, but it also means the signed payload is not bound to the transaction sender or to a specific integration flow.

The function consumes the authorizer's **nonce** and then forwards the authorization update to *Midnight*:

```
require(authorization.nonce == nonce[authorization.authorizer]++, InvalidNonce());

IMidnight(MIDNIGHT)
    .setIsAuthorized(authorization.authorized, authorization.isAuthorized,
authorization.authorizer);
```

If an integration includes a call to *setIsAuthorized()* within a larger transaction, the same signed authorization can be submitted by another party first. The first transaction sets the intended authorization state and consumes the **nonce**. The integration's later transaction then reverts with **InvalidNonce()** when it tries to submit the same signed payload. This is similar to the known frontrunning behavior of ERC-20 *permit()* integrations, which the *MidnightBundles* contract explicitly protects from.

It is recommended to document that *setIsAuthorized()* signatures are permissionless to submit and can be consumed before the intended transaction. Integrations should account for the possible revert.

TRST-R-12 Document that *UtilsLib.msb()* and *tExchange()* expect clean leading bits

Solidity does not guarantee that variables with types shorter than 256 bits have clean higher-order bits when they are read inside inline assembly. And this use of inline assembly is generally discouraged:

"If you access variables of a type that spans less than 256 bits (for example uint64, address, or bytes16), you cannot make any assumptions about bits not part of the encoding of the type. Especially, do not assume them to be zero. To be safe, always clear the data properly before you use it in a context where this is important: uint32 x = f(); assembly { x := and(x, 0xffffffff) / now use x */ } To clean signed types, you can use the signextend opcode: assembly { signextend(<num_bytes_of_x_minus_one>, x) }"*

UtilsLib has two assembly paths that currently rely on narrow typed values already being canonical. First, *msb()* accepts a **uint128 bitmap** and calls *clz()* directly:

```
function msb(uint128 bitmap) internal pure returns (uint256 res) {
    assembly {
        res := sub(255, clz(bitmap))
    }
}
```

Second, *tExchange()* accepts a **bool value** and stores it directly in transient storage:

```
assembly ("memory-safe") {
    previous := tload(slot)
    tstore(slot, value)
}
```

It is recommended to document this behavior for future composition with *UtilsLib*.

TRST-R-13 Code quality improvements

The following code quality improvements are recommended.

In *take()*, post-maturity selling is allowed when no new seller debt is added. The final check enforces this for top-level calls by requiring either zero seller debt, an active liquidation lock, or pre-maturity health:

```
require(
    position[id][seller].debt == 0 || liquidationLocked(id, seller)
        || (block.timestamp <= offer.market.maturity && isHealthy(offer.market, id,
seller)),
    SellerIsLiquidatable()
);
```

However, this check is performed after callbacks, and *take()* sets the seller liquidation lock before invoking those callbacks. During a nested *take()* on the same seller, **liquidationLocked(id, seller)** can therefore be true. This allows the nested call to temporarily increase the seller's debt after maturity. It is recommended to either document this behavior or prevent temporary seller debt after maturity.

Several public getters expose values that are stored as **uint128** in **Position** or **MarketState**, but their return widths are not consistent. For example, **credit**, **debt**, and **totalUnits** are stored as **uint128** but returned as **uint256**:

```
function creditOf(bytes32 id, address user) external view returns (uint256) {
    return position[id][user].credit;
}

function debtOf(bytes32 id, address user) external view returns (uint256) {
    return position[id][user].debt;
}

function totalUnits(bytes32 id) external view returns (uint256) {
    return marketState[id].totalUnits;
}
```

Other values stored as **uint128**, such as **lossFactor**, **pendingFee**, **lastLossFactor**, **collateralBitmap**, and **collateral**, are returned as **uint128**. It is recommended to make the getter return widths consistent.

EcrecoverRatifier.isRatified() and *SetterRatifier.isRatified()* are view functions that verify whether a supplied offer and ratifier data satisfy the ratifier's rules. Both functions currently require **msg.sender == MIDNIGHT**:

```
require(msg.sender == MIDNIGHT, NotMidnight());
```

This restriction makes the functions inaccessible to integrations that want to pre-check whether a specific offer would be ratified. Since these functions do not mutate state and actual offer execution still happens through *Midnight.take()*, allowing arbitrary callers to inspect the result does not weaken the execution path. It is recommended to remove the **msg.sender == MIDNIGHT** requirement.

take() checks that at most one of **offer.maxAssets** and **offer.maxUnits** is nonzero:

```
require(UtilsLib.atMostOneNonZero(offer.maxAssets, offer.maxUnits),
MultipleNonZero());
```

However, the check also accepts offers where both caps are zero. Such offers are not meaningful, since an offer should be capped either by consumed assets or by consumed units. It is recommended to require exactly one nonzero cap.

TRST-R-14 Update inaccurate or incomplete comments

The following comment and NatSpec updates are recommended.

TickLib.wExp() explains the selected **offset** as follows:

```
// offset is chosen such that expR(-offset) == expR(ln2 - offset - 1), so wExp is
non-decreasing.
```

The equality is missing a factor of two. A direct check of the local approximation shows **2 * expR(-offset) == expR(ln2 - offset - 1)**. It is recommended to update the comment:

```
// offset is chosen such that 2 * expR(-offset) == expR(ln2 - offset - 1), so wExp is
non-decreasing.
```

TickLib.priceToTick() documents the expected **spacing** value:

```
/// @dev Among the ticks than are multiples of spacing, returns the lowest one with a
price higher or equal.
/// @dev spacing should divide MAX_TICK.
```

The comment should also state that **spacing** must be greater than zero, otherwise the final division by **spacing** reverts:

```
/// @dev Among the ticks that are multiples of spacing, returns the lowest one with a
price higher or equal.
/// @dev spacing must be greater than zero and should divide MAX_TICK.
```

The *Midnight* offer-cap documentation states:

```
/// @dev It is possible to give units to a fully consumed assets-based buy offer with
price < 1.
```

This can also apply outside the stated buy-offer case, namely for sell offers with zero price. It is recommended to replace the comment with:

```
/// @dev Fully consumed assets-based offers can still be taken for positive units
when the consumed asset amount rounds to zero.
```

The *Midnight* recovery close factor documentation says:

```
/// The maxRepaid computation is rounded up to avoid consecutive max liquidations, so
the position could be slightly
/// healthy after a liquidation.
```

The "avoid consecutive max liquidations" rationale is imprecise, since after the first liquidation the RCF is recalculated. The round-up makes a max-cap liquidation land fractionally on the healthy side, so the outer liquidation condition is closed rather than leaving a wei-scale rounding tail. It is recommended to rephrase this as:

```
/// The maxRepaid computation is rounded up so that a max-cap liquidation lands the
position fractionally on the
/// healthy side, terminating the liquidation chain instead of leaving wei-scale
rounding artifacts.
```

The *Midnight* rounding documentation says:

```
/// @dev Because of roundings, trading and continuous fees might charge less than
expected, which can become problematic
/// for chains where the gas is cheaper than 1 asset of the loan token.
```

This should specify which interactions are relevant, since the gas comparison is not about chain gas in the abstract. It is recommended to clarify whether the concern is repeated *take()* calls, repeated *updatePosition()* calls, or another specific interaction where rounding losses can be harvested, and what the concrete amount of gas is that must not be cheaper than 1 asset.

Finally, *TakeAmountsLib* should document that the asset-target conversion helpers can revert for very large target asset amounts because they use non-full-precision multiplication. For example, offers with **maxAssets = type(uint256).max** are not supported by these helper paths even though the type permits such a value. Add this caveat to *buyerAssetsToUnits()* and *sellerAssetsToUnits()*.

TRST-R-15 Document that ecrecover-based helpers are not compatible with smart wallets

EcrecoverAuthorizer and *EcrecoverRatifier* authenticate signed messages by recovering an address with *ecrecover()*.

This pattern is appropriate for EOAs, but it is not directly compatible with smart wallets. A smart wallet does not produce an ECDSA signature that recovers to the wallet contract address. Instead, smart wallets validate signatures through account-specific logic, such as ERC-1271 *isValidSignature()*.

It is recommended to document that *EcrecoverAuthorizer* and *EcrecoverRatifier* are intended for EOA-style signatures, and to explain the supported alternatives for smart-wallet users. If smart-wallet support is expected to be first-class in the periphery, consider providing an ERC-1271-compatible authorizer and ratifier alongside the existing ecrecover-based contracts.

TRST-R-16 Comments in ratifiers about offer cancellation are incomplete

EcrecoverRatifier and *SetterRatifier* state that “All offers in a tree are expected to share the same maker and ratifier. Otherwise all offers in a tree might not be cancelled by a single call to this function”.

What's missing is that all offers must also uniquely belong to a certain root. Different roots can have overlapping offers.

Centralization risks

TRST-CR-1 Authorized addresses are fully trusted by the authorizer

Users can grant permissions to act on their behalf through the **isAuthorized** mapping. An authorized address has full control over the authorizer's account.

This includes functions that can change the authorizer's position, such as *take()*, *withdraw()*, *repay()*, *supplyCollateral()*, and *withdrawCollateral()*. Authorized addresses can also call *setConsumed()* and *setIsAuthorized()* on behalf of the authorizer. Scoped permission implementations are intentionally left to periphery contracts, keeping *Midnight* itself minimal.

TRST-CR-2 Configurator

The **configurator** is the root administrative role in *Midnight*. It can update the **configurator**, **feeSetter**, **feeClaimer**, and **tickSpacingSetter** addresses. This gives the **configurator** indirect control over all privileged fee, fee-claiming, and tick-spacing configurations.

It can also enable LLTVs and liquidation cursors with *enableLltv()* and *enableLiquidationCursor()*. Disabling LLTVs and liquidation cursors only affects market creation but not existing markets.

TRST-CR-3 Fee setter

The **feeSetter** can configure both market-specific and default fees. It can call *setMarketTradingFee()*, *setDefaultTradingFee()*, *setMarketContinuousFee()*, and *setDefaultContinuousFee()*.

The configured values are bounded by protocol constants (continuous fees are limited to 1% per year, and trading fees to 50bps at 365 days to maturity and decreasing to 0.14 bps closer to maturity), but they still affect market economics. Notably, existing lenders get their continuous fees fixed at *take()* time, leaving them unaffected by later changes.

TRST-CR-4 Fee claimer

The **feeClaimer** can withdraw accumulated protocol fees to an arbitrary receiver. It can call *claimTradingFee()* for **claimableTradingFee** balances and *claimContinuousFee()* for accrued continuous fees.

TRST-CR-5 Tick spacing setter

The **tickSpacingSetter** can call *setMarketTickSpacing()* for existing markets. The new spacing must be nonzero and divide the current spacing, so the role can only keep the current spacing or decrease it to unlock additional ticks while existing ticks remain valid. Since the default tick spacing is **4**, the only allowed transitions are **4 -> 2**, **4 -> 1** and **2 -> 1**.

Changing tick spacing affects which offer ticks are accessible in a market. Lowering the spacing makes more granular prices available, at the expense of fragmenting liquidity. The **tickSpacingSetter** should adjust market granularity only when appropriate and avoid unexpected changes that could affect routers or market makers relying on the previous tick grid.

Systemic risks

TRST-SR-1 External token risks and assumptions

The protocol relies on strong assumptions about all loan tokens and collateral tokens. These are documented directly in the *Midnight* contract.

Since market creation is permissionless, any token can be used to create a market. The risk (e.g., depeg risk, liquidity risk, centralization risk) of external tokens must be assessed on a per-market basis.

The security boundary is the market. One bad token in a market can compromise the whole market but not spread to other markets that don't depend on the bad token.

TRST-SR-2 Oracle risks

Midnight relies on external oracles for collateral valuation. Incorrect, stale, or reverting oracle prices can affect market liveness and pricing. A detailed description of the oracle dependencies, and consequences of liveness and pricing issues, is provided in the *Midnight* contract.

Like for external tokens, the security boundary is the market. It is not possible for a misbehaving oracle in one market to affect another market that doesn't use the same oracle.

TRST-SR-3 Protocol allows for LLTV=WAD with zero collateral buffer

Markets using collateral with **LLTV = WAD** have a special risk profile. They allow borrowing with no overcollateralization buffer, so even small price movements, rounding effects, or liquidation delays can push a position into bad debt.

For such collaterals, the liquidation incentive factor is also **WAD**, meaning liquidators repay at the oracle price plus rounding effects and do not receive an economic liquidation premium. This disables the economic incentive to liquidate positions.

Because there is no collateral buffer, unhealthy positions in **LLTV = WAD** markets will often realize bad debt when liquidated. Lenders in such markets are therefore exposed to oracle error, price movement, execution delays, and liquidation liveness in a qualitatively different way compared to other configurations.